COMP3710 / COMP7710 Report: Pattern Recognition (DRAFT)

Working Prototype Project in the Era of Artificial Intelligence

Course Teaching Team — School of EECS, The University of Queensland

Version 2.4 Final Assessment Specification

Assessment Weighting: 40% of Overall Course Grade **Mode:** Individual Project
Repository: https://github.com/shakes76/PatternAnalysis-2026 **Target Branch:** `topic-recognition`

1 The Commission: Welcome to the Engineering Team

Welcome to the team! Imagine stepping through the doors on your first day as a newly recruited machine-learning engineer in an advanced algorithmic research and development team. Whether in a biomedical imaging laboratory developing next-generation cancer diagnostics, an automated dermatology triage clinic, a social network analytics group, or a quantitative fund building market microstructure simulators, your role is pivotal.

Creating and maintaining algorithms (open or closed source) for pattern recognition and analysis in the current landscape of big data and artificial intelligence (AI) is a foundational capability for any data scientist, software engineer, or researcher. Your technical lead and project manager (represented by your course tutor and the teaching team) calls you in for your initial project commission:

> *"Our organization is tackling several high-impact machine learning initiatives. The problems and associated datasets we are handing you are not toy classroom exercises—they are real-world research and clinical problems actively being solved in laboratories and industry today!*
>
> *We need you to take ownership of a mission-critical recognition challenge. We don't simply need a single aggregate accuracy number—we need you to resolve an authentic engineering dilemma: define the operational requirements, investigate the scientific literature and course concepts, build a functional, modular prototype, benchmark it against an established baseline, rigorously diagnose where and why edge-case failures occur on real data, and deliver an evidence-based recommendation on whether our team should advance this prototype toward deployment or clinical translation."*

In modern industry and research, algorithmic development is dominated by deep learning and built entirely upon open-source software and distributed version control principles. Startups, tech companies, and research institutes do not develop in isolation; they build, test, and contribute to shared codebases using Git workflows.

In this assessment, you will simulate this professional workflow by forking the open-source **PatternAnalysis-2026** repository on GitHub, developing your algorithm on the `topic-recognition` branch, and submitting your solution via a formal GitHub Pull Request. You will complete all the necessary software engineering tasks—including modular architecture design, commenting, testing, and comprehensive documentation in your `README.md` report file. Upon initial submission, the teaching team—experts in the models and domains below—will provide a professional code review of your

pull request a few weeks after the due date. You will then respond to this review by committing adjustments and answering comments to earn dedicated marks, giving you authentic experience of industry code reviews.

1.1 Engineering and Critical Thinking in the Era of AI

We are firmly in the era of artificial intelligence. In your day-to-day engineering work, you have access to powerful generative AI models (such as GitHub Copilot, OpenAI ChatGPT, Anthropic Claude, and Google Gemini) to accelerate boilerplate development, explore unfamiliar APIs, and brainstorm architectural ideas. We actively encourage the professional, responsible use of these tools. The goal is to learn how these models can enhance your learning and engineering workflows, while understanding their critical limitations and failure modes.

However, industry does not hire engineers merely to prompt an LLM and blindly copy-paste its output. In safety-critical applications—such as radiotherapy dose planning, melanoma screening, or financial trading—generative AI models frequently introduce subtle, catastrophic flaws: cross-patient data leakage, uncalibrated confidence scores, anatomically impossible hallucinations, or invalid loss functions.

Your value as an engineer lies in your **critical thinking**, your **mathematical grounding**, your ability to compare your proposed model against an established baseline, your skill in diagnosing edge-case failure modes, and your integrity in auditing every line of code. You may use AI tools to assist your workflow, but you are 100% personally responsible for verifying, validating, and defending every architectural decision, equation, and experimental claim in your report.

1.2 Objective and Difficulty Architecture

Your objective is to solve **ONE** pattern recognition project of your choice from Section 2 using **one** of the suggested model, document your solution and code, and submit it for code review and acceptance into the course repository. To accommodate students with varying technical backgrounds while incentivizing ambitious research, this assessment separates **Implementation Complexity (Difficulty Ceiling)** from **Research Depth (Earned Marks)**:

- **Implementation Difficulty Ceiling (20 Marks):** Determined by the model architecture and data format chosen from the approved tables in Section 2:
  - **[Easy Difficulty]** is capped at **10/20 marks** for the implementation portion of the marking criteria.
  - [Normal Difficulty] is capped at **15/20 marks** for the implementation portion of the marking criteria.
  - **[Hard Difficulty]** is **not capped** for the implementation portion of the marking criteria (eligible for up to **20/20 marks**).
- **Research Depth and Open Dilemma Investigation (Earned Marks):** Regardless of whether you select an Easy, Normal, or Hard model, your primary engineering task is to investigate the domain's **Open Research Question (The Engineering Dilemma)**. Your marks reflect how thoroughly you answer the dilemma: benchmarking against an implemented baseline, conducting qualitative error autopsies on actual failure cases from Rangpur data, and delivering actionable trade-off advice to your project manager. An AI-generated code dump with superficial analysis earns minimal marks; an insightful empirical investigation unlocks the full marks available up to your chosen difficulty ceiling.

Note: The remaining 20 marks of the assessment (Commit Log: 5 marks, Documentation: 10 marks, Pull Request: 5 marks) are common across all difficulty tiers.

1.3 Recommended Progression Pathway

If you are unsure where to start or which difficulty is best suited for you, we recommend an incremental pathway that allows you to start simply, then gradually increase difficulty to improve your marks:

1. Begin with an **[Easy Difficulty]** problem (such as Project 1 on 2D OASIS brain segmentation or Project 2 on 2D HipMRI prostate segmentation). This establishes and verifies your data loading, preprocessing, and training pipeline, securing a working baseline.
2. Once working, scale your solution to **[Normal Difficulty]** (e.g., advancing to HipMRI 2D with an Improved U-Net / CAN, or moving to 3D U-Net) or to **[Hard Difficulty]** (such as 3D volumetric Improved U-Net/CAN3D, VQ-VAE, generative diffusion, ConvNeXt/GFNet, or TimeGAN).

2 Choose an Open-Ended Project

Select ONE model from ONE of the seven open-ended projects below. You must complete all engineering tasks and research guidelines within your chosen project to be eligible for full marks; you do not need to complete multiple projects or multiple models within the same project. Additional recognition problems may become available and will be announced on Blackboard. You are also able to propose your own network for the listed problems or an entirely different problem to the course coordinator for approval.

The teaching staff will decide what constitutes a "reasonably clear image" and the target SSIM is not strict, but both will be negotiable with the staff depending on the difficulty of the problem.

2.1 Classification: Medical Diagnosis and Social Network Analysis

Project Context & Domain Significance

Your manager is evaluating whether an automated classification assistant can assist human reviewers by accurately categorizing incoming cases and reliably routing uncertain predictions for expert review.

- **Neuroimaging (ADNI Brain MRI):** Alzheimer's Disease (AD) causes irreversible neurodegeneration marked by progressive hippocampal and cortical atrophy with ventricular enlargement. Automated classification of T1-weighted structural brain MRIs into AD versus Cognitively Normal (CN) aids early screening. The primary challenge is extracting discriminative anatomical biomarkers amidst significant natural head shape and age variation between patients, while strictly avoiding patient data leakage across multi-slice extractions.
- **Dermoscopic Oncology (ISIC 2020 Melanoma):** Malignant melanoma is the deadliest skin cancer, yet early excision yields high cure rates. Automated screening from dermoscopic lesion images must contend with extreme class imbalance ($< 2\%$ malignant in screening cohorts), visual mimicry between early melanomas and benign atypical nevi, and cutaneous imaging artifacts (hair, ruler markings, air bubbles). Pairwise metric comparison between ambiguous queries and confirmed references provides an effective inductive bias.
- **Graph Mining (Facebook Page-Page Network):** In social and organizational networks, pages represent public entities (politicians, companies, media) linked by mutual likes. Using 128-dimensional descriptive feature embeddings, the goal is semi-supervised multi-class node classification. Graph homophily (interconnected entities often share categories) provides topological context that must be combined with node-level features without label leakage.

The Open Research Question (The Engineering Dilemma)

The Confidence and Ambiguity Dilemma in Automated Triage: In clinical diagnosis (Alzheimer's screening, melanoma detection) and web-scale graph classification, models frequently achieve deceptively high aggregate test accuracy while concealing catastrophic overconfidence on

borderline, ambiguous, or out-of-distribution cases. In safety-critical screening, a single uncalibrated false negative (e.g., misclassifying an early malignant melanoma as benign) or an unflagged ambiguous brain MRI is far more damaging than a routine false positive.

- *Core Investigation:* Does your chosen model architecture reliably quantify uncertainty on ambiguous boundary cases compared to a simple baseline? Does its predicted confidence score (or embedding metric distance) correlate with diagnostic correctness, or does the network remain stubbornly overconfident even when making incorrect predictions?
- *Engineering Recommendation:* Establish an empirical confidence threshold or reject-option rule that safely automates high-confidence cases while routing ambiguous cases to human specialists. Does the added architectural complexity and computational cost of your advanced model justify its operational advantage over the baseline?

Investigate: General Engineering & Research Guidelines

1. **Data Hygiene & Leakage Prevention:** Enforce strict patient-level separation for medical images (all slices/lesions from one patient reside strictly in one split). For graph data, clearly state the transductive or inductive setting and mask test node labels during training and message passing.
2. **Baseline Implementation & Comparison:** Implement a simple baseline model (e.g., a standard convolutional network, MLP, or basic linear classifier) to establish an initial performance benchmark. You will compare your chosen model from the table below against this baseline on the same evaluation data.
3. **Quantitative Benchmarking:** Evaluate beyond raw accuracy. Report per-class precision, recall, macro-F1, and AUROC. For severely imbalanced classes (melanoma), report Precision-Recall curves (PR-AUC). For graph classification, provide a 2D t-SNE or UMAP embedding plot with critical interpretation. Profile computational cost (peak GPU VRAM, parameter count, inference latency).
4. **Failure Case Autopsy & Manager Recommendation:** Inspect 3–5 representative failure cases (false positives and false negatives) from the dataset to diagnose failure triggers. Propose a calibrated operating threshold or decision rule to route low-confidence predictions to human experts, and deliver an actionable recommendation to your project manager.

Model	Difficulty	Dataset, Scope & Key References
Multi-layer GNN	[Normal Difficulty]	Use the Facebook Large Page-Page Network [6] with 128-dim features via PyG. (Note : use the modified Gayanku PyG FacebookPagePage, as the original link is no longer linked to the dataset). Semi-supervised node classification with reasonable accuracy; include t-SNE/UMAP embedding plot with ground truth colors and brief interpretation.
ConvNeXt	[Hard Difficulty]	Use the ADNI dataset (course path `/home/groups/comp3710/ADNI`). Ref: [7]. Modernized convolutional backbone; minimum 0.8 test accuracy on AD vs CN.
GFNet	[Hard Difficulty]	Use ADNI dataset. Ref: [8]. Global Filter Network using 2D FFT token mixing; minimum 0.8 test accuracy on AD vs CN.
Siamese Network	[Hard Difficulty]	Use the ISIC 2020 Kaggle Challenge data. Ref: [9]. Metric learning for melanoma vs normal; pair generation strategy and test accuracy around 0.8.

2.2 Segmentation: Anatomical Contouring and Radiotherapy Planning

Project Context & Domain Significance

Your manager is considering an annotation assistant that proposes anatomical contours for an imaging specialist to review and adjust, saving clinical contouring time.

- **Brain Tissue Segmentation (OASIS):** Delineating cortical gray matter, white matter, and cerebrospinal fluid across 2D slices. Challenges include RF coil intensity inhomogeneity (bias fields), partial volume effects at boundaries, and fine gyral folds.
- **Prostate Radiotherapy Planning (HipMRI):** Targeted external-beam radiotherapy requires millimeter-level delineation of the prostate clinical target volume while tightly sparing adjacent critical Organs at Risk (OARs: bladder, rectum, femoral heads). Under-contouring risks tumor recurrence; over-contouring irradiates rectal mucosa and bladder wall, causing severe radiation toxicity. Multi-parametric 2D slices and full 3D Nifti volumes present soft-tissue contrast ambiguity and extreme class imbalance (prostate volume $< 2\%$ of pelvic scan).
- **2D versus 3D Context Trade-Off:** 2D slice-wise segmentation is memory-efficient and fast, but lacks inter-slice spatial continuity, resulting in jagged 3D boundaries. 3D volumetric models capture continuous anatomical geometry across slices, but demand high GPU memory and careful subvolume/patch management.

The Open Research Question (The Engineering Dilemma)

The Spatial Context vs. Clinical Boundary Utility Dilemma: Standard global Sørensen-Dice similarity coefficients (≥ 0.75 or ≥ 0.90) often mask critical local boundary failure modes. In prostate radiotherapy planning (HipMRI), an algorithm can achieve an acceptable global Dice score across wide central slices while failing catastrophically at the superior (base) and inferior (apex) slices, or leaking across the millimeter-thin interface into the rectal wall or bladder mucosa. In clinical radiotherapy, boundary leakage causes severe radiation proctitis or cystitis, whereas under-contouring risks lethal cancer recurrence. In brain tissue segmentation (OASIS), high global Dice can coexist with topologically invalid disconnections in fine cortical gyri.

- *Core Investigation:* Does incorporating advanced multi-scale context (CAN) or true 3D volumetric continuity (3D U-Net / 3D Improved U-Net / CAN3D) genuinely eliminate these hazardous boundary and slice-transition errors compared to a standard 2D baseline (e.g., 2D U-Net)? Or does it merely smooth easy central slices while demanding 5–10$\times$ the GPU memory and training time?
- *Engineering Recommendation:* Pinpoint anatomically where spatial context delivers measurable clinical gain and where it breaks down. Deliver an evidence-based recommendation to your project manager on whether 3D volumetric context or multi-scale dilation justifies the computational budget in an operational contouring pipeline.

Investigate: General Engineering & Research Guidelines

1. **Data Pipeline & Alignment:** Enforce strict patient-level splitting. Standardize intensity normalization and one-hot multi-channel mask encoding. Preserve exact spatial alignment between images and ground truth masks (see Appendix B for Nifti loading).
2. **Baseline Implementation & Comparison:** Implement a standard baseline model (e.g., a basic 2D or 3D U-Net) to establish an initial performance benchmark. You will compare your chosen model from the table below against this baseline on the same evaluation split, evaluating segmentation accuracy and boundary precision.
3. **Quantitative Benchmarking:** Report per-class Sørensen-Dice similarity coefficients (DSC) and Intersection-over-Union (IoU) on the held-out test split. Profile peak GPU VRAM and per-case inference time.
4. **Failure Case Autopsy & Usability Analysis:** Provide visual multi-panel comparisons: input image, ground truth mask, predicted mask, and difference overlays (false positives and false negatives).

Inspect 3–5 specific boundary failure cases (prostate base and apex slices, small disconnected artifacts). Advise your manager which anatomical cases remain unsuitable for automated drafting.

Model	Difficulty	Dataset, Scope & Key References
2D U-Net	[Easy Difficulty]	Use HipMRI 2D slices (`keras_slices_data`). Ref: [1]. Target Dice ≥ 0.75 on prostate label on test set. Requires loading Nifti format (see Appendix B).
2D Improved U-Net	[Easy Difficulty]	Use OASIS Brain 2D dataset. Ref: [2]. Target Dice ≥ 0.9 across all tissue labels on test set.
2D CAN	[Easy Difficulty]	Use OASIS Brain 2D dataset. Ref: [3]. Dilated context aggregation; target Dice ≥ 0.9 across all labels on test set.
2D Improved U-Net	[Normal Difficulty]	Use HipMRI 2D slices. Ref: [2]. Target Dice ≥ 0.75 on prostate label on test set.
2D CAN	[Normal Difficulty]	Use HipMRI 2D slices. Ref: [3]. Context aggregation network; target Dice ≥ 0.75 on prostate label on test set.
3D U-Net	[Normal Difficulty]	Use downsampled HipMRI 3D data. Ref: [4]. Volumetric convolutions and skips; target Dice ≥ 0.7 on test set.
3D Improved U-Net	[Hard Difficulty]	Use downsampled HipMRI 3D data. Ref: [2]. Volumetric residual units and deep supervision; target Dice ≥ 0.7 on test set.
CAN3D	[Hard Difficulty]	Use downsampled HipMRI 3D data. Ref: [5]. 3D dilated context aggregation; target Dice ≥ 0.7 on test set. Use pyimgaug3d or PyTorch 3D transforms.

2.3 Lesion Detection: Dermoscopic Screening and Triage

Project Context & Domain Significance

In primary clinical screening, dermoscopic images capture uncropped, variable-scale skin fields with multiple benign lesions, marker ink, and hairs alongside suspicious lesions. Rather than assuming pre-cropped inputs, an operational triage system must detect, localize with bounding boxes, and classify skin lesions simultaneously. Key challenges include wide lesion scale variance (tiny macules to large spreading patches), low border contrast against surrounding skin, and clinical sensitivity requirements where missing a malignancy is far more costly than flagging a benign nevus for inspection.

The Open Research Question (The Engineering Dilemma)

The Clinical Sensitivity vs. Biopsy Overload Trade-Off: In primary dermatological screening, automated detection systems face an acute operational dilemma. Setting a low confidence threshold detects subtle, early-stage, or faint melanomas (maximizing sensitivity/recall), but triggers excessive false alarms on benign atypical nevi, cutaneous ruler markings, ink stains, and hair follicles. In clinical practice, excessive false positives overwhelm dermatology clinics with unnecessary biopsies, escalating healthcare costs and patient distress. Conversely, raising the detection threshold reduces false alarms but risks missing life-threatening malignant melanomas.

- *Core Investigation:* How does your chosen detection model (e.g., YOLOv7 or newer) navigate the precision-recall frontier compared to a standard baseline detector? What specific visual triggers (contrast variation, hair artifacts, peripheral border fuzziness) deceive the bounding-box localization or class prediction?
- *Engineering Recommendation:* Formulate an empirical multi-tier operating policy for your project manager (e.g., automated clear for high-confidence benign cases, mandatory urgent biopsy for detected malignancies, and intermediate monitoring for uncertain detections). Quantify the trade-off between biopsy burden and detection sensitivity at your recommended operating point.

Investigate: General Engineering & Research Guidelines

1. **Data Pipeline & Bounding Boxes:** Convert mask annotations into standard bounding box coordinates ($[x_{min}, y_{min}, x_{max}, y_{max}]$ or normalized $[x_c, y_c, w, h]$). Maintain patient-independent validation and test splits.
2. **Baseline Implementation & Comparison:** Implement an initial baseline detection model (e.g., a standard or default detector configuration) to establish a performance benchmark. You will compare your chosen model from the table below against this baseline on the held-out test split, analyzing localization precision and recall.
3. **Quantitative Evaluation:** Evaluate on held-out test data using standard detection metrics: mean Average Precision (mAP_{50}, $mAP_{50:95}$), Precision, Recall, and per-class localization IoU (target IoU ≥ 0.8). Profile inference throughput (FPS) for mobile/clinic feasibility.
4. **Failure Case Autopsy & Operating Point:** Visualize predicted bounding boxes and confidence scores against ground truth. Inspect 3–5 representative failure cases (false alarms from ink marks or hairs, missed small macules). Formulate and justify an optimal clinical operating threshold to your project manager.

Model	Difficulty	Dataset, Scope & Key References
YOLOv7 or newer	[Normal Difficulty]	Use ISIC 2017/2018 data (`/home/groups/comp3710/ISIC2018`). Pre-trained models allowed; target test IoU ≥ 0.8 and suitable accuracy for classification.

2.4 Image Generation: Synthetic Medical Cohorts and Privacy

Project Context & Domain Significance

Medical imaging research is constrained by stringent patient data privacy laws (HIPAA, GDPR) and limited clinical scan availability. Generative models allow synthesizing realistic synthetic cohorts for data augmentation and anonymized multi-centre sharing. However, healthcare synthesis carries unique hazards: the model must preserve authentic physiological tissue boundaries without mode collapse, and must not hallucinate anatomically impossible structures or memorize training scans (which would violate patient confidentiality).

The Open Research Question (The Engineering Dilemma)

The Fidelity vs. Memorization vs. Privacy Dilemma in Synthetic Medicine: Synthesizing synthetic medical cohorts promises to bypass strict patient confidentiality laws (HIPAA, GDPR) and expand scarce clinical datasets for open research. However, generative architectures face a dangerous trilemma: (1) mode collapse or blurry anatomical representations, (2) hallucinating physiologically impossible tissue junctions or non-existent pathologies, and (3) verbatim memorization of training patient scans. If a generative model simply memorizes training scans, sharing the synthetic dataset constitutes an illegal breach of patient confidentiality; if it hallucinates unrealistic tissue structures, downstream models trained on it will learn hazardous diagnostic artifacts.

- *Core Investigation:* Does your chosen generative architecture (DCGAN, WGAN-GP, VQ-VAE, VQ-VAE2, StyleGAN, or Stable Diffusion) produce genuinely novel, diverse, and anatomically coherent scans, or does it memorize patient training examples or collapse into limited modes? How does it compare against a simpler generative baseline in structural fidelity (SSIM > 0.6) and distributional coverage?
- *Engineering Recommendation:* Conduct an empirical nearest-neighbor distance audit and latent space interpolation between synthetic images and training scans. Deliver an evidence-based verdict

to your project manager on whether the generated cohort is safe, non-memorized, and clinically plausible for downstream training.

Investigate: General Engineering & Research Guidelines

1. **Pipeline Setup & Training Stability:** Implement appropriate loss formulations (e.g., Wasserstein distance with gradient penalty, discrete codebook quantization, or latent diffusion schedules). Address training stability and gradient hygiene.
2. **Baseline Implementation & Comparison:** Implement an initial generative baseline (e.g., standard DCGAN or basic autoencoder) to establish a benchmark. Compare your chosen model from the table below against this baseline on sample clarity, diversity, and fidelity.
3. **Quantitative Benchmarking:** Compute Structural Similarity Index (SSIM > 0.6) across synthesized cohorts against reference anatomical slices. Measure distribution fidelity (FID, feature coverage, or latent distance metrics). Profile GPU memory, training stability, and generation runtime.
4. **Qualitative Autopsy & Memorization Audit:** Display uncurated sample grids across diverse random seeds (avoid cherry-picking). For style-based or diffusion models, provide a 2D t-SNE or UMAP embedding plot comparing real and synthetic distributions. Conduct a failure autopsy on 3–5 flawed generations (checkerboard artifacts, distorted tissue margins, mode drops) and audit nearest training neighbors to disprove patient memorization.

Model	Difficulty	Dataset, Scope & Key References
DCGAN	[Normal Difficulty]	Use HipMRI 2D slices (`keras_slices_data`). Ref: [10]. Reasonably clear image with SSIM > 0.6.
WGAN / WGAN-GP	[Normal Difficulty]	Use HipMRI 2D slices. Ref: [11] (recommended). Critic with Lipschitz constraint; reasonably clear image with SSIM > 0.6.
VQ-VAE	[Hard Difficulty]	Use HipMRI 2D slices. Ref: [12]. Discrete codebook vector quantization; reasonably clear image with SSIM > 0.6.
VQ-VAE2	[Hard Difficulty]	Use HipMRI 2D slices. Ref: [13]. Hierarchical multi-scale latent quantization; clear image with SSIM > 0.6.
StyleGAN / StyleGAN2	[Hard Difficulty]	Use ADNI brain MRI. Refs: [14, 15, 16]. Style modulation; clear image, t-SNE/UMAP embedding plot with ground truth colors. Note: Requires heavy GPU resources.
Stable Diffusion	[Hard Difficulty]	Use ADNI brain MRI. Ref: [17]. Latent diffusion fine-tuning; clear image, t-SNE/UMAP embedding plot with ground truth colors. Note: Requires heavy GPU resources.

2.5 Image Colourisation: State Space Restoration

Project Context & Domain Significance

Converting single-channel greyscale images into natural, plausible color representations is an under-constrained, ill-posed inverse problem: a single greyscale intensity corresponds to multiple legitimate real-world hues. High-quality colourisation enhances human perceptual interpretability in scientific, clinical, and archival imaging. Selective State Space Models provide linear-time sequence modeling with global receptive fields, offering an alternative to computationally expensive self-attention.

The Open Research Question (The Engineering Dilemma)

Plausible Synthesis vs. Ground-Truth Illusion in State Space Restoration: Converting single-channel greyscale images into natural RGB representations is mathematically ill-posed: a single greyscale luminance intensity corresponds to an infinite manifold of legitimate real-world hues. Standard pixel-wise regression losses (MSE/L1) inevitably produce drab, desaturated, brownish outputs because

the network minimizes expected error by predicting the conditional mean of the color distribution. Furthermore, models can produce convincing yet completely ungrounded chromatic hallucinations (e.g., unnaturally colored skin or organs).

- *Core Investigation:* When fine-tuning a modern selective state space model (MambaIRv2), does the architecture's linear-time global receptive field enable it to synthesize vivid, structurally coherent colors that strictly respect semantic boundaries, or does it suffer from chromatic bleeding and unnatural splotches? How does its restored quality compare against an initial baseline (e.g., untuned model or simple color mapping)?
- *Engineering Recommendation:* How do you rigorously demonstrate to your project manager that a synthesized colourisation is 'plausible' when ground-truth chromaticity is fundamentally unrecoverable from greyscale alone? Provide a trade-off analysis evaluating perceptual quality (LPIPS, Colorfulness, SSIM) against GPU memory footprint and fine-tuning throughput.

Investigate: General Engineering & Research Guidelines

1. **Pipeline Setup & Distributed Execution:** Set up the environment using the provided Colab starter notebook. Configure reproducible fine-tuning scripts utilizing `torch.run`.
2. **Baseline Implementation & Comparison:** Establish an initial evaluation baseline (e.g., evaluating the frozen pretrained model or a standard color mapping baseline). Compare your fine-tuned model against this baseline on the supplied test set.
3. **Quantitative Benchmarking:** Evaluate on the supplied unseen test set. Report PSNR, SSIM, and perceptual metrics (LPIPS, Colorfulness index). Document GPU memory, fine-tuning epochs, and training time.
4. **Qualitative Autopsy & Plausibility Diagnostics:** Present side-by-side comparisons: Greyscale Input, Baseline Output, Fine-tuned Output, and Ground Truth. Conduct an autopsy on 3–5 representative failure cases (color bleeding across high-contrast edges, desaturation collapses, chromatic hallucinations). Advise your project manager on the operational boundaries of automated colourisation.

Model	Difficulty	Dataset, Scope & Key References
MambaIRv2	[Hard Difficulty]	1. Adapt the structure and fine tune a already pre-trained Mamba modelIRv2 to do GreyScale → Color images 2. You can use the starter notebook if needed. 3. You can use / make / re-purpose **any** dataset suitable for the fine tuning 4. You need to get the model completed, fine tuned, tested with any dataset that need and make it ready for inference (colorization) 5. Few days before the submission date, we will provide some (test) samples images in grey scale. You will need to colorized these and include in the report.

2.6 Radiology Report Summarisation: Clinical Language Models

Project Context & Domain Significance

Clinical radiology reports are written in dense, specialized anatomical and pathological language intended for physicians. When patients read these reports via online electronic health portals, they frequently experience confusion or anxiety. The engineering goal is to translate technical findings into plain, empathetic, layperson-accessible summaries (∼6th-grade reading level). In healthcare NLP, generation errors carry severe safety risks: hallucinating non-existent pathologies, omitting acute

incidental findings (e.g., pneumothorax, nodules), or distorting clinical certainty.

The Open Research Question (The Engineering Dilemma)

The High ROUGE vs. Clinical Safety Dilemma in Layperson Summarisation: Translating complex clinical radiology reports into layperson-accessible language (∼6th-grade reading level) is crucial for patient autonomy and reducing healthcare anxiety. However, standard natural language generation metrics—such as ROUGE-1, ROUGE-2, and ROUGE-L—measure surface n-gram lexical overlap, not clinical veracity. A language model can achieve an outstanding ROUGE score by generating fluent, empathetic prose while committing fatal clinical errors: hallucinating a non-existent tumor, omitting an acute incidental finding (such as an early pneumothorax or critical pulmonary nodule), or altering diagnostic certainty from 'suspected' to 'confirmed'.

- *Core Investigation:* Does fine-tuning your chosen model (T5, FLAN-T5, or GPT-2 via LoRA vs. full fine-tuning) genuinely preserve factual clinical fidelity compared to an untuned or zero-shot baseline, or does it merely improve superficial linguistic fluency?
- *Engineering Recommendation:* Perform a rigorous clinical safety audit on 3–5 representative test cases: categorise errors into hallucinations, critical omissions, and layperson readability trade-offs. Formulate an evidence-based recommendation for your project manager detailing what automated clinical guardrails (e.g., entity verification, confidence filtering) must be integrated before any patient-facing deployment.

Investigate: General Engineering & Research Guidelines

1. **Data Pipeline & Tokenization:** Parse dataset splits without data leakage. Implement proper sequence truncation, target sequence masking, and special task prompts.
2. **Baseline Implementation & Comparison:** Establish an initial baseline summary generation (e.g., zero-shot / pre-adaptation outputs or a standard summarization baseline). You will compare your fine-tuned model from the table below against this baseline on the validation split.
3. **Quantitative Benchmarking:** Evaluate on the held-out validation split using standard summarization metrics: ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-Lsum. Document total and trainable parameter counts, peak VRAM, and inference throughput (tokens/second).
4. **Clinical Safety Autopsy (Mandatory Task):** Select 3–5 representative test cases and perform a qualitative safety audit: (a) Factual faithfulness (hallucinations), (b) Information completeness (omissions of acute findings), and (c) Layperson readability. Explain why ROUGE scores alone do not guarantee clinical safety, and advise your manager on deployment risks.

Model	Difficulty	Dataset, Scope & Key References
T5 / FLAN-T5	**[Hard Difficulty]**	Use BioLaySumm LaymanRRG dataset [19, 20] (ACL 2025). Encoder-decoder; evaluate ROUGE scores (rouge1, rouge2, rougeL, rougeLsum), LoRA vs full fine-tuning, report GPU/VRAM/time, and 3–5 example clinical error analysis. Heavy GPU resources / `torch.run`.
GPT-2	**[Hard Difficulty]**	Use BioLaySumm LaymanRRG dataset [21]. Autoregressive decoder; target-masked loss, generation/stopping criteria, ROUGE scores, and error analysis. Heavy GPU resources / `torch.run`.

2.7 Time-Series Generation: Limit Order Book and Market Simulation

Project Context & Domain Significance

In quantitative finance, evaluating trading algorithms, execution strategies, and risk systems requires realistic market simulations. Real microsecond Limit Order Book (LOB) data is commercially restricted and expensive. Generative time-series models synthesize realistic order book event sequences. Unlike generic time series, generated order books must strictly respect market microstructure invariants: spread validity ($P_{bid,1} < P_{ask,1}$), monotonic price ladders across levels, non-negative liquidity depth, and realistic stylized facts such as volatility clustering.

The Open Research Question (The Engineering Dilemma)

The Microstructure Invariant and Arbitrage Dilemma in Market Simulation: Simulating high-frequency Limit Order Book (LOB) dynamics is essential for stress-testing execution algorithms and market risk systems without paying prohibitive market data licensing fees. However, unlike generic continuous time series, financial order books must strictly conform to hard economic and physical microstructure laws: strictly positive bid-ask spread ($P_{bid,1} < P_{ask,1}$), strictly monotonic price ladders across all 10 order depth levels, non-negative depth volumes, and stylized temporal facts (volatility clustering, heavy-tailed return distributions). Generic time-series models often produce visually plausible return curves that produce crossed order books (creating instantaneous risk-free arbitrage opportunities) or completely violate volume conservation.

- *Core Investigation:* Does TimeGAN's joint supervised temporal loss and unsupervised adversarial loss actually learn and enforce the hard structural constraints of the order book compared to a standard baseline generator? Does it reproduce authentic volatility clustering, or does it collapse into white noise or mode collapse?
- *Engineering Recommendation:* Conduct an in-depth microstructure audit on 3–5 simulation periods: quantify the frequency of crossed-book violations, compute KL divergence on spreads/returns (≤ 0.1), and evaluate depth heatmap structural similarity (SSIM > 0.6). Advise your project manager whether the generated synthetic order books are reliable for backtesting algorithmic trading strategies.

Investigate: General Engineering & Research Guidelines

1. **Data Pipeline & Chronological Integrity:** Apply log-returns transformation for prices and volume normalization. Enforce strict chronological splitting (train on earlier market hours, test on later hours) to eliminate look-ahead bias.
2. **Baseline Implementation & Comparison:** Implement an initial baseline model to establish a time-series benchmark. Compare your trained model from the table below against this baseline on distribution fidelity and microstructure validity.
3. **Quantitative Benchmarking:** Compute Kullback-Leibler (KL) divergence (target ≤ 0.1) between real and synthetic distributions for spread and mid-price returns. Compute Structural Similarity (SSIM > 0.6) on 2D depth snapshots (price levels $\times$ time steps). Report GPU memory, training stability, and throughput.
4. **Microstructure Diagnostics & Error Analysis (Mandatory Task):** Provide 3–5 representative heatmap visualizations comparing real vs. synthetic depth surfaces. Diagnose structural failures: Did the model produce crossed books ($P_{bid} \geq P_{ask}$)? Does it reproduce volatility clustering or collapse into uncorrelated noise? Formulate a clear recommendation for your project manager.

Model	Difficulty	Dataset, Scope & Key References
TimeGAN	[Hard Difficulty]	Use LOBSTER dataset (AMZN Level 10 data). Generative time-series model; target KL divergence ≤ 0.1 on spread and midprice returns, SSIM > 0.6 on depth heatmaps, report GPU/VRAM/time, and 3–5 heatmap visualizations with error analysis. Heavy GPU resources / `torch.run`.

3 Required Feasibility Check-Off (Compulsory Midpoint Hurdle)

This is a compulsory, ungraded assessment hurdle. You must complete it during a scheduled practical consultation by the published project midpoint (check exact course profile for dates).

As a newly recruited engineer, you must present a **one-page written feasibility review** to your project manager (tutor) before expending extensive computing resources or committing to an unviable architecture. The review must address the open engineering dilemma of your chosen domain and cover:

1. **User Need, Scope & Acceptance Criteria:** The intended user, prototype purpose, and 3–5 testable acceptance criteria addressing the domain's open dilemma, including an operational/resource constraint (e.g., peak VRAM, runtime).
2. **Model Choice & Course Concepts:** The proposed dataset configuration, model architecture, chosen difficulty band, and how course concepts and literature support the choices.
3. **Preliminary Feasibility Evidence:** An initial data audit, baseline smoke-test experiment, or measured resource estimate. A fully trained final model is not required.
4. **Risks, Budget & Fallback:** Key technical risks, computing budget, the next planned experiment, and a workable fallback.

Consultation Format: Your tutor reads your one-page review, provides brief verbal feedback, and records completion (approximately 3–5 minutes). **No live demonstration, presentation slides, or oral examination is required.** Responding to or adopting the feedback is optional and carries no separate consultation marks.

Assessment Hurdle Status: Check-off confirms that a complete review was presented and discussed. **If the check-off is not completed, you cannot pass this assessment**, subject to approved university adjustment and make-up arrangements. No numerical marks are awarded or deducted during the practical consultation.

Inclusion in Final Report: Include your written feasibility review as a clearly identified section in your final report (`README.md`). Its technical quality is marked as part of your final report documentation.

4 Submission and Codebase Architecture

The submission of the final report will be done via a GitHub pull request and a Turnitin submission:

1. The code of your algorithm and test script(s) written to run the algorithm (20 Marks).
2. Commit log to provide evidence of originality (5 Marks, Pass Hurdle).
3. Documentation via a `README.md` file (10 Marks).
4. GitHub Pull Request and Code Review response (5 Marks, Pass Hurdle).

The Turnitin submission should be a PDF export of the `README.md` file. **You are required to make the pull request to receive any marks for the assessment. You must complete a pull request (with a working version of the algorithm committed) at least once before the assessment deadline.**

Code Review Timeline: Submitting your pull request by the deadline completes Phase 1. A few weeks after the assessment due date, the teaching team will post formal code review feedback directly

on your GitHub pull request. Actively addressing and incorporating this feedback is marked under Component 4 (see Section 5.4).

4.1 Mandatory Codebase File Structure

Your algorithm must reside in a dedicated folder under the `recognition/` directory of the Pattern-Analysis trunk with a meaningful name. We suggest using a name that incorporates the name of your model appended with your name or student ID (e.g., `recognition/Improved_UNet_s41234567/`).

To ensure clean software engineering design and modularity, your implementation **must include the following five files**:

1. **`modules.py`:** Containing the source code of the components of your model. Each component must be implemented as a class or a function in PyTorch or TensorFlow/Keras. Your implementation of the model should not depend on NumPy (or other Python libraries not part of PyTorch / TensorFlow) in any way unless otherwise approved by the teaching staff.
2. **`dataset.py`:** Containing the data loader for loading and preprocessing your data, including leakage-free train/validation/test splitting functions and data augmentations.
3. **`train.py`:** Containing the source code for **training**, **validating**, **testing**, and **saving** your model. The model should be imported from `modules.py` and the data loader should be imported from `dataset.py`. Make sure to plot the losses and metrics during training.
4. **`predict.py`:** Showing example usage of your **trained** model. It should load your saved model weights, run inference on test cases, print out any results, and provide visualisations where applicable (e.g., prediction overlays, generated samples, heatmaps). NumPy is allowed in this script for visualization or loading data.
5. **`README.md`:** Sufficiently documenting your project, working principles, feasibility review, experiments, and instructions (see Section 5.3).

Note: You may create other helper files such as `utils.py` to better organise your project. You may reuse some of your own elements from a previous demo, but the final model must be substantially different from your previous work.

Strict Git Repository Rules:

- **Do not commit datasets or model files.** Your fork and commits should only contain code/scripts plus any additional visual content needed for your `README.md` file. Committing multi-gigabyte files will result in pull-request rejection.

5 Marking Criteria: 40 Marks Total

The assessment is marked out of **40 marks** according to the official four-part course marking scheme:

Assessment Component	**Assessment Focus**	**Marks Allocation**
1. Recognition Problem (Implementation)	Code Quality & Methodological Rigor	20 Marks
2. Commit Log	Evidence of Progressive Originality	5 Marks (Pass Hurdle)
3. Documentation (`README.md`)	Technical Report & Critical Evaluation	10 Marks
4. GitHub Pull Request	Open Source Contribution & Code Review	5 Marks (Pass Hurdle)
Total		**40 Marks**

5.1 1. Recognition Problem (Implementation) — 20 Marks

Your implementation will be assessed on the following six criteria:

1. Algorithm solves the respective pattern recognition problem appropriately (5 Marks).
2. Algorithm implemented in TF/PyTorch demonstrated and functions as intended to solve the respective problem (3 Marks).
3. Good design implemented in TF/PyTorch that solves the problem (1 Mark).
4. Commenting and code readability (1 Mark).
5. Algorithm is normal or above difficulty while implemented in TF/PyTorch while solving the respective problem (5 Marks).
6. Algorithm is hard difficulty while implemented in TF/PyTorch while solving the respective problem (5 Marks).

Difficulty Capping Rules on Implementation:

- **[Easy Difficulty]** is capped at **10/20 marks** for the implementation portion of the marking criteria (criteria 1–4 eligible; criteria 5 and 6 capped at 0).
- **[Normal Difficulty]** is capped at **15/20 marks** for the implementation portion marking criteria (criteria 1–5 eligible; criterion 6 capped at 0).
- **[Hard Difficulty]** is **not capped** for the implementation portion marking criteria (eligible for all 20/20 marks across criteria 1–6).

5.2 2. Commit Log — 5 Marks (Pass Hurdle)

Ensure that when you are implementing your algorithm to solve the recognition problem, you are committing regularly and keeping a detailed commit log. You should, for example, commit whenever you reach any small milestone, such as creating a module, writing a working data loader, completing the algorithm, running an experiment, or completing documentation.

1. Meaningful commit messages with evidence of individual work and originality (2 Marks).
2. Progressive commits and logical log structure providing evidence of individual work and originality (3 Marks).

Pass Hurdle Warning: You must have a reasonable commit log showing progressive development of your algorithm. You should not commit your entire submission as one commit. You will not receive any marks if your algorithm only contains a single commit. You may not pass the assessment if your commit log does not provide sufficient evidence of individual work and originality. Dumping a single AI-generated codebase in one commit fails this hurdle.

5.3 3. Documentation via `README.md` — 10 Marks

Once your algorithm has been implemented, you will need to provide sufficient comments in your code and comprehensive documentation of your algorithm in a `README.md` file. The readme file should contain:

- A title, a description of the algorithm and the problem that it solves (approximately a paragraph), how it works in a paragraph, and a figure/visualisation.
- Your one-page written feasibility review from Section 3.
- Any dependencies required, including exact versions, and address reproducibility of results.
- Example inputs, outputs, and visual plots of your algorithm.
- Description of any specific pre-processing used, with references if any. Justify your training, validation, and testing splits of the data.
- **Investigation of the Open Research Dilemma:**
  - Quantitative benchmarking of your chosen model against your implemented baseline model under identical evaluation splits.

  - Resource profiling table documenting computational efficiency (peak GPU VRAM, parameter count, and inference runtime/latency).
  - In-depth qualitative error autopsies analyzing 3–5 representative failure cases from the dataset (diagnosing root triggers and failure modes).
  - Actionable engineering recommendation and trade-off synthesis for your project manager.
- The mandatory **Artificial Intelligence Usage Disclosure** subsection (see Section 6).

Marking Criteria:

1. Description and explanation of the working principles of the algorithm implemented and the problem it solves (5 Marks).
2. Description of usage and comments throughout scripts (3 Marks).
3. Proper formatting using GitHub markdown (2 Marks).

5.4 4. GitHub Pull Request and Code Review — 5 Marks (Pass Hurdle)

Lodge a pull request to the PatternAnalysis-2026 repository via the `topic-recognition` branch. A pull request must be made to receive any marks for this assessment, with a working version of the algorithm committed.

The GitHub Pull Request assessment operates across two distinct chronological phases:

- **Phase 1: Initial Submission (by Assessment Due Date):** Open your pull request against upstream `topic-recognition`. Your PR must include your working modular code and comprehensive `README.md`, accompanied by a professional pull request title and description.
- **Phase 2: Code Review Feedback & Response (Few Weeks After Due Date):** Approximately 2–3 weeks after the initial submission due date, members of the teaching team will review your pull request and post detailed, professional code review feedback, questions, and suggested modifications directly on your GitHub PR thread.

Marking Criteria (5 Marks Total):

1. Creating a pull request into the correct branch with a working and demonstrable version of the algorithm by the due date (2 Marks).
2. **Incorporating feedback into the pull request from code review (2 Marks): There are dedicated marks allocated for addressing and incorporating code review feedback.** You are required to actively respond to the reviewers' feedback on GitHub. To earn these 2 marks, you must read the reviewer comments, commit requested code modifications or clarifications to your branch (which automatically updates the open PR), and reply to reviewer comments on GitHub. A designated response window (e.g., 7 days from feedback release) will be provided. Failing to respond to reviewer feedback during this window forfeits these 2 marks.
3. Professional description of and comments within the pull request body (1 Mark).

Guides on creating pull requests via GitHub can be found here and here. See also this tutorial by Atlassian.

6 Academic Integrity, Engineering Ethics, and AI Disclosure

Academic Integrity is a core value of the UQ community, and high academic integrity expectations apply to assessments. Using online resources for self-learning is fine; however, **simply re-using entire blocks of code from them is not acceptable and may result in a violation of the Student Charter as a UQ student and an investigation into Academic Misconduct**. You must follow the guidelines for Academic Integrity found here, which also applies to code.

Where possible, you should write your own code; any elements adapted from online resources or literature must be explicitly cited in your pull-request, code docstrings, and `README.md`. For more information see the UQ policy here and the official UQ tutorial.

6.1 Responsible Use of AI and Mandatory Disclosure

Generative AI models are permitted to assist your learning and implementation workflows. However, the expectations of engineering professionalism apply:

1. **Verification Responsibility:** You are 100% personally responsible for every line of code, loss formula, and metric in your submission. Blaming an AI model for an incorrect calculation or data leak is unacceptable.
2. **Mandatory AI Disclosure Section:** In your `README.md`, you must include a dedicated subsection titled **"Artificial Intelligence Usage Disclosure"** documenting:
   - Which AI tools were utilized (e.g., GitHub Copilot for auto-complete, ChatGPT/Claude for debugging syntax, Gemini for concept lookup).
   - What specific tasks AI assisted with (e.g., scaffolding data loading functions, refactoring plotting scripts).
   - How you audited, verified, and empirically tested the AI-suggested code.
3. **Prohibited Misconduct:** Copying code from fellow students, reusing previous years' pull requests, or committing unverified AI hallucinations.

7 Tips and Practical Advice

- **Start with an Easy Problem First:** It is best advised to try an Easy problem first, especially if you are not so confident with TF/PyTorch and the problems provided. Once working, expand up to a Normal problem such as the recommended pathway mentioned in Section 1.3.
- **Generative Models Advice:** For generative problems, try to get your code working on a simpler dataset such as MNIST first and then attempt it with the data for your problem.
- **Review Previous Years' Pull Requests:** You should inspect previous years' successful pull requests here (2024) and here (2025), as well as the type of expected content, code style, and hints to the problems.
- **Hard Problems Warning: Hard problems should only be attempted if you are extremely confident with your deep learning skills!**
- **Rangpur HPC Compute Resources:** Compute-intensive tasks (3D models, StyleGAN, Stable Diffusion, Mamba, LLMs) should be executed on the Rangpur HPC cluster (`/home/groups/comp3710/`). Information and details for Rangpur can be found here. Use interactive GPU sessions for testing and Slurm batch scripts for longer runs.

8 Checklist

The following steps should be followed to complete this assessment task:

1. Fork the PatternAnalysis-2026 library on GitHub.
2. Checkout the `topic-recognition` branch within your personal fork.
3. Choose **ONE** model from one project via Section 2 and create a folder for your problem in the `recognition/` folder of the PatternAnalysis trunk with a meaningful name (e.g., incorporating the name of your model appended with your name or student ID).
4. Prepare your one-page written feasibility review and complete the compulsory practical check-off consultation with your tutor by the project midpoint (Section 3).
5. Create a `README.md` file that documents your report, working principles, feasibility review, and instructions.
6. Complete the solution to the recognition problem by building your model (`modules.py`, `dataset.py`, `train.py`, `predict.py`), making sure to commit any progress regularly to the `topic-recognition` branch of your fork.
7. Do not commit datasets or model files; your fork and commits should only have code/scripts plus any additional visual content needed for your `README.md` file.

8. Complete the test driver script (`predict.py`) that calls and runs your trained algorithm.
9. Complete the documentation of your algorithm, including a summary of your results, baseline comparison, error analysis, and model in your `README.md` file.
10. Lodge a pull request to upstream `topic-recognition` before the submission deadline (Phase 1).
11. Complete a Turnitin submission of the PDF export of the `README.md` file.
12. A few weeks after the deadline, review teaching team code review feedback on your GitHub pull request, commit requested improvements, and reply to comments to secure your code review marks (Phase 2).

9 Conclusion

By completing this report, you will obtain invaluable experience in developing and maintaining your own open-source project, how to contribute to other open-source projects via pull requests, and good practices in algorithmic design and implementation. You will learn how to operate as a machine learning engineer in the era of AI—using cutting-edge AI tools to accelerate your workflow while applying rigorous critical thinking, mathematical verification, and failure analysis to solve real-world problems. All the while, you will become even more knowledgeable in PyTorch/TensorFlow and modern pattern recognition!

Appendix

A Datasets

The following datasets are all available on the Rangpur HPC in the *`/home/groups/comp3710`* directory. The information and details for Rangpur can be found at this link. You can also find these datasets at the sources below:

1. **OASIS Brain data set (9K images):** This is part of the OASIS brain study with segmentation labels. The preprocessed version of this data set can be found on Rangpur at:

   ```
   Rangpur Path: /home/groups/comp3710/OASIS
   ```

2. **HipMRI Study for prostate cancer radiotherapy:** Consisting of Prostate 2D MRI slice data (under the `keras_slices_data` folder) for 2D tasks and Prostate 3D MRI data for 3D tasks of 38 patients with 211 3D MRI volumes with MRIs in the `semantic_MRs` folder and the segmentations in the `semantic_labels_only` folder. The body outline, bone, bladder, rectum, and prostate regions are segmented in 3D by an MR physicist with more than 10 years of experience.

   ```
   Rangpur Path: /home/groups/comp3710/HipMRI_Study_open
   2D Data:      /home/groups/comp3710/HipMRI_Study_open/keras_slices_data
   ```

3. **ADNI dataset for Alzheimer's disease:** Structural MRI scans available from `http://adni.loni.usc.edu`. The preprocessed version has two classes: Alzheimer's disease (AD) and Cognitive Normal (CN). It can be found on Rangpur at:

   ```
   Rangpur Path: /home/groups/comp3710/ADNI
   ```

4. **ISIC 2017/2018 dermoscopic imaging challenge data for skin cancer:** Part of the ISIC challenge with segmentation labels and lesion class labels:

   ```
   Rangpur Path: /home/groups/comp3710/ISIC2018
   ```

5. **ISIC 2020 Kaggle Challenge data set:** Dermoscopic imaging with image-level class labels (normal or melanoma). See also the ISIC 2020 Challenge and Kaggle page.
6. **Facebook Large Page-Page Network dataset:** Graph dataset with 128-dimensional node feature vectors available through Gayanku PyG FacebookPagePage.
7. **BioLaySumm 2025 ACL workshop dataset:** Subtask 2.1 LaymanRRG open-source track for layperson translation of radiology reports. Dataset and downloads: Hugging Face. Workshop overview: biolaysumm.org.
8. **LOBSTER dataset (AMZN level 10 data):** High-frequency limit order book event data. Link: LOBSTER Data Samples.

B Reading Nifti Files

The following code blocks are examples of loading Nifti image files using the Nilearn and Nibabel libraries, which are the standard file format in medical imaging. Listing 1 shows how to load 2D Nifti files and Listing 2 shows how to load 3D Nifti files as NumPy arrays.

Listing 1: Loading 2D Medical Nifti Images

```
import numpy as np
import nibabel as nib
from tqdm import tqdm

def to_channels(arr: np.ndarray, dtype=np.uint8) -> np.ndarray:
    channels = np.unique(arr)
    res = np.zeros(arr.shape + (len(channels),), dtype=dtype)
    for c in channels:
        c = int(c)
        res[..., c:c+1][arr == c] = 1

    return res

# load medical image functions
def load_data_2D(imageNames, normImage=False, categorical=False, dtype=np.
    float32, getAffines=False, early_stop=False):
    '''
    Load medical image data from names, cases list provided into a list for
        each.

    This function pre-allocates 4D arrays for conv2d to avoid excessive
        memory usage.

    normImage: bool (normalise the image 0.0-1.0)
    early_stop: Stop loading pre-maturely, leaves arrays mostly empty, for
        quick loading and testing scripts.
    '''
    affines = []

    # get fixed size
    num = len(imageNames)
    first_case = nib.load(imageNames[0]).get_fdata(caching='unchanged')
    if len(first_case.shape) == 3:
        first_case = first_case[:,:,0] # sometimes extra dims, remove
    if categorical:
        first_case = to_channels(first_case, dtype=dtype)
        rows, cols, channels = first_case.shape
        images = np.zeros((num, rows, cols, channels), dtype=dtype)
    else:
        rows, cols = first_case.shape
        images = np.zeros((num, rows, cols), dtype=dtype)
```

```
38
39      for i, inName in enumerate(tqdm(imageNames)):
40          niftiImage = nib.load(inName)
41          inImage = niftiImage.get_fdata(caching='unchanged') # read disk only
42          affine = niftiImage.affine
43          if len(inImage.shape) == 3:
44              inImage = inImage[:,:,0] # sometimes extra dims in HipMRI_study
                    data
45          inImage = inImage.astype(dtype)
46          if normImage:
47              inImage = (inImage - inImage.mean()) / (inImage.std() + 1e-8)
48          if categorical:
49              inImage = to_channels(inImage, dtype=dtype)
50              images[i,:,:,:] = inImage
51          else:
52              images[i,:,:] = inImage
53
54          affines.append(affine)
55          if i > 20 and early_stop:
56              break
57
58      if getAffines:
59          return images, affines
60      else:
61          return images
```

Listing 2: Loading 3D Medical Nifti Images

```
1  def load_data_3D(imageNames, normImage=False, categorical=False, dtype=np.
       float32, getAffines=False, orient=False, early_stop=False):
2      '''
3      Load medical image data from names, cases list provided into a list for
           each.
4
5      This function pre-allocates 5D arrays for conv3d to avoid excessive
           memory usage.
6
7      normImage: bool (normalise the image 0.0-1.0)
8      orient: Apply orientation and resample image? Good for images with large
           slice thickness or anisotropic resolution
9      dtype: Type of the data. If dtype=np.uint8, it is assumed that the data
           is labels
10     early_stop: Stop loading pre-maturely? Leaves arrays mostly empty, for
           quick loading and testing scripts.
11     '''
12     affines = []
13
14     interp = 'linear'
15     if dtype == np.uint8: # assume labels
16         interp = 'nearest'
17
18     # get fixed size
19     num = len(imageNames)
20     niftiImage = nib.load(imageNames[0])
21     first_case = niftiImage.get_fdata(caching='unchanged')
22     if len(first_case.shape) == 4:
23         first_case = first_case[:,:,:,0] # sometimes extra dims, remove
24     if categorical:
25         first_case = to_channels(first_case, dtype=dtype)
26         rows, cols, depth, channels = first_case.shape
27         images = np.zeros((num, rows, cols, depth, channels), dtype=dtype)
```

```
    else:
        rows, cols, depth = first_case.shape
        images = np.zeros((num, rows, cols, depth), dtype=dtype)

    for i, inName in enumerate(tqdm(imageNames)):
        niftiImage = nib.load(inName)
        inImage = niftiImage.get_fdata(caching='unchanged') # read disk only
        affine = niftiImage.affine
        if len(inImage.shape) == 4:
            inImage = inImage[:,:,:,0] # sometimes extra dims in
                HipMRI_study data
        inImage = inImage[:,:,:depth] # clip slices
        inImage = inImage.astype(dtype)
        if normImage:
            inImage = (inImage - inImage.mean()) / (inImage.std() + 1e-8)
        if categorical:
            inImage = to_channels(inImage, dtype=dtype)
            images[i,:inImage.shape[0],:inImage.shape[1],:inImage.shape
                [2],:] = inImage
        else:
            images[i,:inImage.shape[0],:inImage.shape[1],:inImage.shape[2]]
                = inImage

        affines.append(affine)
        if i > 20 and early_stop:
            break

    if getAffines:
        return images, affines
    else:
        return images
```

References

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.

[2] F. Isensee, P. Kickingereder, W. Wick, M. Bendszus, and K. H. Maier-Hein, "Brain tumor segmentation and radiomics survival prediction: Contribution to the BRATS 2017 challenge," in *Brainlesion: Glioma, Multiple Sclerosis, Stroke and TBI Models*, 2018, pp. 287–297.

[3] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," in *Proc. ICLR*, 2016.

[4] Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger, "3D U-Net: Learning dense volumetric segmentation from sparse annotation," in *Proc. MICCAI*, 2016, pp. 424–432.

[5] W. Dai, S. Chandra, et al., "Context aggregation network for 3D medical image segmentation," 2021.

[6] B. Rozemberczki, C. Allen, and R. Sarkar, "Multi-scale attributed node embedding," *Journal of Complex Networks*, vol. 9, no. 2, 2021.

[7] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. CVPR*, 2022, pp. 11976–11986.

[8] Y. Rao, W. Zhao, Z. Zhu, J. Lu, and J. Zhou, "Global filter networks for image classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 7837–7850, 2023.

[9] G. Koch, R. Zemel, and R. Salakhutdinov, "Siamese neural networks for one-shot image recognition," in *ICML Deep Learning Workshop*, 2015.

[10] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. ICLR*, 2016.

[11] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. ICML*, 2017, pp. 214–223.

[12] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Proc. NeurIPS*, 2017.

[13] A. Razavi, A. van den Oord, and O. Vinyals, "Generating diverse high-fidelity images with VQ-VAE-2," in *Proc. NeurIPS*, 2019.

[14] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," in *Proc. CVPR*, 2019, pp. 4401–4410.

[15] T. Karras et al., "Analyzing and improving the image quality of StyleGAN," in *Proc. CVPR*, 2020, pp. 8110–8119.

[16] S. Liu, S. Chandra, et al., "Style-based generative models for medical neuroimaging," *IEEE Transactions on Medical Imaging*, 2023.

[17] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. CVPR*, 2022, pp. 10684–10695.

[18] H. Guo et al., "MambaIRv2: Attentive state space restoration," *arXiv preprint arXiv:2501.00000*, 2025.

[19] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[20] H. W. Chung et al., "Scaling instruction-finetuned language models," *arXiv preprint arXiv:2210.11416*, 2022.

[21] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, 2019.